CHAPTER 6 — LAB TUTORIAL

Vue.js — Building an Interactive Profile Card

A hands-on lab using Vue 3 (Composition API) from CDN

Cross-Platform Application Development

Duration: 3 Hours (Lecture + Lab) | Version: Vue 3.x

1. Lab Overview

In this lab, you will build a responsive, interactive profile-card web page using Vue 3. You will practice every core concept from today's lecture: creating a Vue app instance, reactive state with ref(), the core directives (v-bind, v-on, v-model, v-if, v-for), and computed properties.

1.1 Learning Outcomes

By the end of this lab you will be able to:

- Bootstrap a Vue 3 application from a CDN with a single HTML file.
- Declare reactive state using ref() inside setup().
- Render data to the page using mustache interpolation {{ }}.
- Apply the core directives v-bind, v-on, v-model, v-if, and v-for.
- Use a computed property to derive a value from state.
- Toggle a CSS theme (dark mode) using :class binding.

1.2 Prerequisites

- Basic HTML, CSS, and JavaScript (ES6+).
- A modern browser (Chrome, Edge, or Firefox).
- A code editor (Visual Studio Code recommended, with the Vue-Official extension).
- No Node.js or installation required — we use the CDN build of Vue.

1.3 What you will build

A single-page profile card that:

- Shows a user's name, title, and short bio, editable in real time.
- Lists the user's skills with the ability to add and remove items.
- Displays a live skill count using a computed property.
- Toggles between a light and dark theme with one click.

> **Estimated time**
> About 90 minutes of guided work, plus 20 minutes for the bonus challenges at the end.

---

2. Environment Setup

Step 1 — Create a new project folder

1. Create a new folder named vue-lab-ch6 on your desktop.
2. Open the folder in Visual Studio Code (File → Open Folder…).
3. Inside the folder, create a new file called index.html.

Step 2 — Install a live preview extension (optional but recommended)

4. In VS Code, open the Extensions panel (Ctrl + Shift + X).

5. Search for "Live Server" by Ritwick Dey and click Install.
6. Once installed, right-click index.html and choose Open with Live Server. Your browser will auto-refresh every time you save.

Step 3 — Starter HTML

Paste the following skeleton into index.html. This is a plain HTML page (no Vue yet) to confirm your editor and browser are working:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Vue Lab – Profile Card</title>
</head>
<body>
  <h1>Hello, world!</h1>
</body>
</html>
```

Save the file and open it in the browser. You should see the "Hello, world!" heading.

> **Checkpoint**
>
> If the page does not load, confirm the file name is exactly index.html and that Live Server is running on the correct folder.

---

3. Activity 1 — Create the Vue Application (15 min)

Replace the body of index.html so that Vue is loaded from the CDN and a Vue app is created. We will not add any features yet — just confirm Vue is running.

3.1 Load Vue 3 from CDN

Add this <script> tag just before the closing </body> tag:

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

3.2 Add a mount point

Replace the <h1> element with a that has id="app". This is where Vue will render:

```
<div id="app">
  <h1>{{ greeting }}</h1>
</div>
```

3.3 Create the Vue app

Immediately below the Vue CDN <script> tag, add a second <script> block that creates and mounts your app:

```
<script>
  const { createApp, ref } = Vue

  createApp({
    setup() {
      const greeting = ref('Hello from Vue 3!')
      return { greeting }
    }
  }).mount('#app')
</script>
```

Save the file and refresh the browser. You should see the text "Hello from Vue 3!".

> **Self-check**
>
> Open DevTools (F12) and inspect the page. The <h1> should contain the actual text "Hello from Vue 3!", not the literal mustache syntax.
>
> If you see {{ greeting }} as raw text on the page, Vue did not mount — check for JavaScript errors in the console.

---

4. Activity 2 — Build the Profile Card (25 min)

Now let us add real profile data and render it to the page using interpolation and v-bind.

4.1 Replace your mount template

Replace the contents of with the following:

```
<div id="app" :class="{ dark: darkMode }">
  <section class="card">
    <img :src="avatar" :alt="name" class="avatar">
    <h2>{{ name }}</h2>
    <p class="title">{{ title }}</p>
    <p class="bio">{{ bio }}</p>
  </section>
</div>
```

4.2 Update your Vue setup()

```
<script>
  const { createApp, ref } = Vue

  createApp({
    setup() {
      const name     = ref('Nur Aisyah')
      const title    = ref('Full-Stack Developer')
      const bio      = ref('Loves Vue, coffee, and open-source.')
```

```
      const avatar   = ref('https://i.pravatar.cc/200?img=47')
      const darkMode = ref(false)

      return { name, title, bio, avatar, darkMode }
    }
  }).mount('#app')
</script>
```

4.3 Add CSS styling

Add the following <style> block inside <head> — it styles the card and defines both a light and a dark theme:

```
<style>
  body { font-family: system-ui, sans-serif; background: #F5F7FA;
         margin: 0; padding: 2rem; transition: background .3s; }
  #app { max-width: 420px; margin: 0 auto; }
  .card {
    background: #FFFFFF; border-radius: 16px; padding: 2rem;
    box-shadow: 0 8px 24px rgba(0,0,0,.08); text-align: center;
    transition: background .3s, color .3s;
  }
  .avatar { width: 120px; height: 120px; border-radius: 50%;
            border: 4px solid #42B883; }
  .title  { color: #42B883; font-weight: 600; margin: .25rem 0 1rem; }
  .bio    { color: #64748B; font-size: .95rem; }

  /* Dark theme */
  .dark            { }
  .dark .card      { background: #1E293B; color: #F1F5F9; }
  .dark body, body.dark { background: #0F172A; }
  .dark .bio       { color: #94A3B8; }
</style>
```

Save and reload. You should see a neat profile card with the avatar, name, title, and bio.

> **What just happened**
>
> {{ name }} renders the reactive value inside the <h2>.
>
> :src="avatar" is v-bind shorthand — it dynamically binds the HTML src attribute to the reactive avatar ref.
>
> :class="{ dark: darkMode }" adds the CSS class dark only when darkMode.value is true. We will wire it up next.

---

5. Activity 3 — Make It Editable with v-model (15 min)

Two-way binding lets users edit the profile and see changes instantly.

5.1 Add an edit form

Below the <section class="card"> in your template, add this form block:

```
<section class="editor">
  <h3>Edit Profile</h3>

  <label>Name:
    <input v-model="name" type="text">
  </label>

  <label>Title:
    <input v-model="title" type="text">
  </label>

  <label>Bio:
    <textarea v-model="bio" rows="3"></textarea>
  </label>

  <label>
    <input type="checkbox" v-model="darkMode"> Dark mode
  </label>
</section>
```

5.2 Style the editor

Add this CSS inside your existing <style> block:

```
.editor {
  background: #FFFFFF; padding: 1.5rem; border-radius: 16px;
  margin-top: 1rem; box-shadow: 0 4px 12px rgba(0,0,0,.06);
}
.editor h3 { margin-top: 0; color: #35495E; }
.editor label { display: block; margin: .75rem 0; font-size: .9rem; }
.editor input[type=text], .editor textarea {
  width: 100%; padding: .5rem; border: 1px solid #CBD5E1;
  border-radius: 6px; font-size: 1rem; margin-top: .25rem;
}
.dark .editor { background: #1E293B; color: #F1F5F9; }
```

Save and refresh. Type into the Name, Title, or Bio fields — the profile card above updates as you type. Check the Dark mode box and the page should instantly switch to dark theme.

> **Why does this work?**
>
> Each <input v-model="…"> sets up a two-way binding: changes to the input update the reactive ref, and vice versa. Because the card also reads those same refs, they stay in sync automatically.

6. Activity 4 — Skills List with v-for (20 min)

6.1 Add skills state

Back in <script>, add a new reactive array and an input ref inside setup():

```
const skills    = ref(['Vue.js', 'JavaScript', 'HTML & CSS'])
const newSkill  = ref('')

function addSkill() {
  const s = newSkill.value.trim()
  if (!s) return
  if (!skills.value.includes(s)) {
    skills.value.push(s)
  }
  newSkill.value = ''
}

function removeSkill(index) {
  skills.value.splice(index, 1)
}

return {
  name, title, bio, avatar, darkMode,
  skills, newSkill, addSkill, removeSkill
}
```

Don't forget to return!

Anything you want to use in the template must be returned from setup(). If "skills is not defined" appears, check the return object.

6.2 Add the skills UI

Inside the <section class="card"> element (after the <p class="bio">), add:

```
<ul class="skills">
  <li v-for="(skill, i) in skills" :key="skill">
    {{ skill }}
    <button @click="removeSkill(i)" title="Remove">&times;</button>
  </li>
</ul>
```

Then inside the editor section, add a small form to add a new skill:

```
<label>Add a skill:
  <input v-model="newSkill"
         @keyup.enter="addSkill"
         placeholder="e.g. Node.js">
</label>
<button class="btn" @click="addSkill">+ Add skill</button>
```

6.3 Style the skills

```
.skills { list-style: none; padding: 0; margin: 1rem 0 0; }
.skills li {
  display: inline-flex; align-items: center;
  background: #ECFDF5; color: #065F46; padding: .3rem .7rem;
  border-radius: 999px; margin: .25rem; font-size: .85rem;
}
.skills li button {
  background: none; border: none; color: #065F46;
  margin-left: .5rem; cursor: pointer; font-size: 1rem;
}
.btn {
  background: #42B883; color: white; border: none;
  padding: .6rem 1rem; border-radius: 6px; cursor: pointer;
  font-weight: 600;
}
.btn:hover { background: #33A06F; }
.dark .skills li { background: #064E3B; color: #6EE7B7; }
```

Save, reload, and try adding/removing skills. Press Enter in the input to add — that's the .enter key modifier on v-on.

---

7. Activity 5 — Computed Property & Conditional Rendering (10 min)

7.1 Import computed

At the top of your <script>, update the destructuring line:

```
const { createApp, ref, computed } = Vue
```

7.2 Derive skillCount

Inside setup(), add a computed property that returns the number of skills:

```
const skillCount = computed(() => skills.value.length)

// remember to add skillCount to the return object
return {
  name, title, bio, avatar, darkMode,
  skills, newSkill, addSkill, removeSkill,
  skillCount
}
```

7.3 Render it with v-if / v-else

Below the skills list in your card, add:

```
<p v-if="skillCount === 0" class="muted">
```

```
  No skills yet — add your first skill to get started.
</p>
<p v-else class="muted">
  Listed <strong>{{ skillCount }}</strong>
  skill<span v-if="skillCount !== 1">s</span>.
</p>
```

Add a tiny style:

```
.muted { color: #94A3B8; font-size: .85rem; margin-top: .75rem; }
```

Reload the page. Remove all skills one by one — the message changes from "Listed 3 skills." to "No skills yet…". Add one back and it reads "Listed 1 skill." (no s). That's the combined power of computed, v-if, and v-else.

> **computed vs method**
>
> You could also write {{ skills.length }} directly in the template. We use computed here to see how derived values work: Vue caches the result and only re-computes when skills changes.

---

8. Activity 6 — Wrap-up & Final Test (5 min)

Your final index.html should now be a self-contained, single-file Vue app with roughly 120 lines of code. Run the following quick checks:

7. Type a new name — does the <h2> update instantly?
8. Toggle Dark mode — does the whole page switch theme?
9. Add three skills, press Enter after each — do they appear as green pills?
10. Click × on a pill — does the skill disappear and the count update?
11. Remove all skills — does the "No skills yet" message appear?

---

9. Bonus Challenges (Optional)

Finished early? Try these extensions — each one reinforces a different directive or concept.

Challenge A — Watchers

- Import watch from Vue.
- Log the old and new value each time name changes.
- **Hint:** `watch(name, (n, o) => console.log(o, '->', n))`

Challenge B — Form validation

- Disable the + Add skill button when newSkill is empty.

- Hint: use `:disabled="!newSkill.trim()"`

Challenge C — Persist to localStorage

- On page load, read an existing profile from localStorage (if any).
- Whenever the name, bio, or skills change, write the profile back to localStorage.
- Hint: use watch with { deep: true } on an object containing the three values.

Challenge D — Upload a new avatar

- Add <input type="file" @change="onAvatarChange">.
- In the handler, read the file with FileReader and set avatar.value to a data-URL.

— End of Chapter 6 Lab —